Hacettepe University — Computer Engineering
CMP 720 EMBEDDED SYSTEMS DESIGN

Spring 2026 — Mini In-Class Project Series Part 4

Lab 4: RTOS Migration, Scheduling & Priority Inversion

SafePulse — Embedded Patient Monitoring System

Week 5: System Software, RTOS, Scheduling & Real-Time Properties

Duration:	1 – 1.5 hours
Tools Required:	Renode, Zephyr SDK, terminal, Week 2 spec
Deliverable:	Multi-task RTOS firmware + priority inversion observation
Prerequisites:	Completed Lab 3 (Week 4) — working I2C sensor driver

1 Overview

This week you port SafePulse from a bare-metal architecture to a **multi-task RTOS** design using Zephyr. You will decompose the monolithic loop into three cooperating tasks with different priorities, communicate between them via a message queue, add cycle-accurate instrumentation to measure scheduling and alarm latencies, and—most importantly—**observe what happens when priorities are wrong**.

> **The key lesson this week:** A correct algorithm with **incorrect priority assignment** can harm a patient just as surely as a missing alarm. RTOS scheduling is not optional complexity—it is a safety mechanism.

1.1 Learning Objectives

1. Decompose a monolithic system into cooperating RTOS tasks with clear responsibilities.

2. Use Zephyr kernel primitives: threads, message queues (`k_msgq`), mutexes, semaphores, timers, and `k_poll()`.

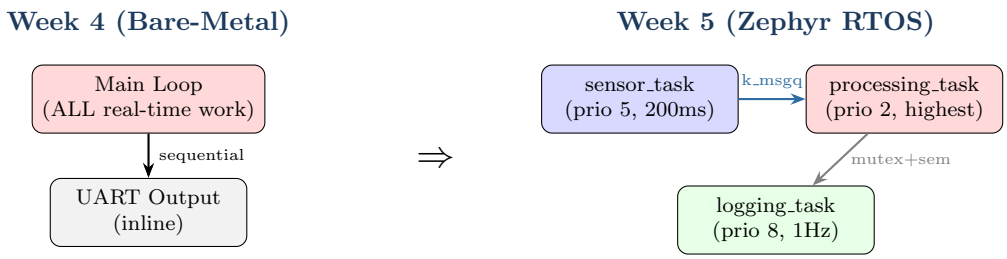
3. Separate lock-free safety-critical paths from mutex-protected non-critical paths.

4. Assign task priorities correctly based on safety criticality analysis.

5. Instrument scheduling and alarm latencies using `k_cycle_get_32()`.

6. Observe priority inversion and understand its consequences in safety-critical systems.

1.2 Architecture: Week 4 vs. Week 5



1.3 Design Principles

The Week 5 architecture separates the **alarm path** from the **log path** within the processing task:

- **Alarm path (lock-free):** When sensor data arrives, the processing task evaluates thresholds and writes the alarm GPIO *without acquiring any mutex and without calling printk*. This path cannot be blocked by any other task.
- **Log path (mutex-protected):** *After* the alarm GPIO is written, the processing task acquires `log_mutex` to update the snapshot that the logging task will read. If the logging task currently holds the mutex, the delay only affects log delivery—the alarm has *already* fired.

This separation guarantees that alarm latency depends *only* on scheduling delay (sensor post → processing start) and threshold evaluation time—never on UART or mutex contention.

2 Zephyr RTOS Concepts

2.1 Tasks (Threads)

In Zephyr, a **thread** is an independently scheduled execution context with its own stack. Threads are defined statically with `K_THREAD_DEFINE()` or dynamically with `k_thread_create()`.

2.2 Priorities

Zephyr uses a **lower number = higher priority** convention. Preemptive priorities (≥ 0) allow higher-priority threads to preempt lower-priority ones at any point. This means that if the `processing_task` has priority 2 and the `logging_task` has priority 8, the processing task will *always* preempt the logging task when it has work to do.

> **Why this matters for SafePulse:** If a critical SpO2 reading arrives while the logging task is in the middle of a slow `printk()`, the processing task **preempts it immediately**, processes the alarm, and returns control to the logging task afterward. The alarm fires with minimal delay regardless of what non-critical work is happening.

2.3 Message Queues

`k_msgq` provides a fixed-size FIFO for passing structured data between threads. The sender calls `k_msgq_put()` and the receiver calls `k_msgq_get()`. The receiver can block (`K_FOREVER`) until a message arrives—the scheduler will run other threads during the wait.

2.4 Kernel Timers

`k_timer` provides precise periodic wakeups. The sensor task uses `k_timer_start(&sensor_timer, K_MSEC(200), K_MSEC(200))` to create a 200 ms cadence, then calls `k_timer_status_sync()` to block until the next expiry. Unlike `k_msleep(200)`, the timer approach does not accumulate drift from I2C transaction time: the period is measured from expiry to expiry, not from "end of work to end of work."

2.5 Mutexes and Semaphores

A **mutex** (`k_mutex`) provides mutual exclusion for shared data. In our design, `log_mutex` protects the log snapshot variables that the processing task writes and the logging task reads. A **semaphore** (`k_sem`) is used as a one-shot notification: the processing task calls `k_sem_give()` on state transitions, and the logging task can wake immediately instead of waiting for its next timer tick.

2.6 Polling (`k_poll`)

`k_poll()` lets a thread wait on *multiple* events simultaneously. The logging task polls on two events: (1) the state-change semaphore, and (2) a timer-driven poll signal. Whichever fires first wakes the thread. This is the Zephyr equivalent of `select()`/`epoll()` in POSIX, and it lets the logging task respond immediately to state transitions while still maintaining its regular 1 Hz (or 2 Hz) cadence.

> **Why does a "non-critical" task get event-driven wakeup?** The alarm GPIO—the safety-critical output—fires in the lock-free path of the processing task, completely independent of the logging task. What the logging task adds is *clinical context*: the LED tells the nurse "something is wrong," but the UART message says "SpO2 dropped to 85%, heart rate is 42." Without the semaphore, that text could arrive up to 1 s after the LED, leaving the nurse staring at a blinking light with no detail. The semaphore closes this gap to milliseconds. This is a **usability optimization, not a safety requirement**—the alarm is already active regardless.

2.7 Cycle-Accurate Timing

`k_cycle_get_32()` reads the hardware cycle counter (DWT on Cortex-M4). Combined with `k_cyc_to_us_floor32()`, it lets us measure sub-microsecond intervals. The sensor task stamps the cycle count just before posting to the message queue; the processing task reads the cycle count when it wakes. The difference is the **scheduling delay**—how long the message waited in the queue before being serviced.

2.8 Provided Project Structure

File	Purpose
`CMakeLists.txt`	Zephyr build configuration
`prj.conf`	Kernel configuration (multithreading, GPIO, UART, `CONFIG_POLL=y`)
`main.c`	Skeleton with 4 TODOs (priorities + 3 task bodies)
`main_priority_inversion.c`	Pre-built buggy version for Part 2 experiment (shared in the lab)
`stm32f4.repl` not provided	We include default `stm32f4_discovery.repl` in `.resc`
`vitals_sensor.py`	I2C sensor hooks (IronPython, same as Week 4)
`safepulse.resc`	Renode script (loads Zephyr ELF)

prj.conf must include `CONFIG_POLL=y` for `k_poll()` support. Without it, the logging task will fail to compile. The provided `prj.conf` already has this set.

2.9 Build and Run

Build with: `west build -b stm32f4_disco . --pristine`
Run with: `renode safepulse.resc`

3 Part 1: Correct RTOS Implementation

3.1 Task 1: Assign Priorities

The skeleton defines three priority constants, all set to 5. Assign correct priorities based on the criticality of each task. Remember: **lower number = higher priority** in Zephyr.

Listing 1: TODO 1: Priority assignment

```
#define PROCESSING_PRIORITY   5   /* TODO: Safety-critical alarm */
#define SENSOR_PRIORITY       5   /* TODO: Periodic sampling */
#define LOGGING_PRIORITY      5   /* TODO: Non-critical display */
```

3.2 Task 2: Implement `sensor_task`

This task uses a kernel timer at 200 ms cadence, reads the sensor via I2C, stamps the cycle count for latency measurement, and sends data to the processing task via the message queue.

Listing 2: TODO 2: Implement `sensor_task_fn`

```
/* ============================================================
 * TODO 2: Implement sensor_task
 *
 * This task is timer-driven at 200ms using k_timer_status_sync().
 * Unlike k_msleep(), the timer approach keeps precise cadence
 * regardless of how long the I2C reads take.
 *
 * Steps (inside the loop):
```

```
 *   a) Wait for sensor_timer to expire: k_timer_status_sync(&sensor_timer)
 *   b) Read sensor ID via i2c_read_register() to check presence
 *   c) If OK (ID == 0xA5), read HR/SpO2/Temp into vitals_msg_t, set sensor_ok=1
 *      If not, memset msg to 0, set sensor_ok=0
 *   d) Set msg.sample_id
 *   e) Stamp cycle count: msg.sensor_cycles = k_cycle_get_32()
 *   f) Send via k_msgq_put(&vitals_msgq, &msg, K_NO_WAIT)
 *   g) Increment sample_id
 *
 * IMPORTANT: Start the timer BEFORE the loop with:
 *   k_timer_start(&sensor_timer, K_MSEC(200), K_MSEC(200));
 *
 * Why K_NO_WAIT on put? If the queue is full (processing task slow),
 * we drop the message rather than blocking---missing one 200ms sample
 * is safer than stalling the entire sampling loop.
 * ============================================================ */
void sensor_task_fn(void *p1, void *p2, void *p3) {
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);
    uint32_t sample_id = 0;

    /* TODO: Start the sensor timer here */

    while (1) {
        /* TODO: Your code here */

        sample_id++;
    }
}
```

3.3 Task 3: Implement processing_task

This is the safety-critical core. It blocks on the message queue and, when data arrives, executes two distinct paths:

1. **Alarm path (lock-free):** Evaluate thresholds and write the alarm GPIO. No mutex, no printk, no blocking calls.
2. **Log path (mutex-protected):** After the alarm fires, acquire `log_mutex` and update the log snapshot for the logging task.

Listing 3: TODO 3: Implement `processing_task_fn`

```
/* ============================================================
 * TODO 3: Implement processing_task
 *
 * This is the safety-critical core. The alarm path MUST be
 * lock-free: threshold check -> GPIO write, with NO mutex,
 * NO printk, NO blocking calls.
 *
 * Steps (inside the loop):
 *   a) Block on k_msgq_get(&vitals_msgq, &msg, K_FOREVER)
```

```
 *   b) Record scheduling delay:
 *         uint32_t proc_start = k_cycle_get_32();
 *         uint32_t sched_delay_us = k_cyc_to_us_floor32(proc_start - msg.sensor_cycles);
 *
 *   --- ALARM PATH (lock-free, no UART, no blocking) ---
 *   c) Determine new_state from sensor data:
 *      - sensor_ok == 0 -> STATE_SENSOR_FAULT
 *      - CRITICAL: SpO2<90 OR temp>40.0 OR HR<40 OR HR>150
 *      - WARNING:  SpO2<=94 OR temp>=38.0 OR HR<50 OR HR>120
 *      - Otherwise: STATE_NORMAL
 *   d) Fire alarm GPIO immediately:
 *      - CRITICAL or SENSOR_FAULT: alarm_on()
 *      - Transitioning OUT of CRITICAL/FAULT: alarm_off()
 *      - Measure alarm_latency_us after GPIO write
 *   e) Record proc_time_us, update current_state
 *
 *   --- LOG PATH (mutex protected, AFTER alarm fires) ---
 *   f) Lock log_mutex, write log_vitals/log_state/timing fields, unlock
 *   g) If state changed, notify logging: k_sem_give(&state_change_sem)
 *
 * WHY this separation matters: The alarm fires within microseconds
 * of data arrival. The mutex (for log data) is acquired AFTER the
 * GPIO write, so even if the logging task holds the mutex, the alarm
 * is never delayed.
 * ============================================================ */
void processing_task_fn(void *p1, void *p2, void *p3) {
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    while (1) {
        /* TODO: Your code here */
    }
}
```

3.4 Task 4: Implement `logging_task`

The logging task is the *sole UART writer*. It wakes on *either* a periodic timer or a state-change semaphore, whichever comes first, using `k_poll()`.

Listing 4: TODO 4: Implement `logging_task_fn`

```
/* ============================================================
 * TODO 4: Implement logging_task
 *
 * This task is the SOLE UART writer. It wakes on EITHER:
 *   - A periodic timer (1Hz normal, 2Hz in warning/critical), OR
 *   - A state-change semaphore (for immediate transition reports)
 * using k_poll() to wait on both events simultaneously.
 *
 * Steps:
 *   a) Track prev_state (initialized to STATE_NORMAL)
 *   b) Set up k_poll_event array (provided below as a hint)
 *   c) Start the log timer: k_timer_start(&log_timer, K_MSEC(1000), K_MSEC(1000))
```

```
 *   d) Inside the loop:
 *      1. k_poll(events, 2, K_FOREVER)
 *      2. Reset whichever event(s) fired:
 *         - Semaphore: k_sem_take(&state_change_sem, K_NO_WAIT), reset state
 *         - Signal: k_poll_signal_reset(&log_timer_signal), reset state
 *      3. Lock log_mutex, snapshot all log_* variables, unlock
 *      4. If state changed from prev_state, print transition message
 *         (e.g., "[ALARM] CRITICAL: SpO2=...") and update prev_state
 *      5. Print regular vitals line with timing diagnostics:
 *         [timestamp][sample_id] HR=... [sched=X us, proc=Y us, alarm=Z us]
 *      6. Adjust timer rate: 500ms for warning/critical, 1000ms for normal
 *
 * HINT --- k_poll event setup:
 *   struct k_poll_event events[2] = {
 *       K_POLL_EVENT_INITIALIZER(K_POLL_TYPE_SEM_AVAILABLE,
 *                                K_POLL_MODE_NOTIFY_ONLY,
 *                                &state_change_sem),
 *       K_POLL_EVENT_INITIALIZER(K_POLL_TYPE_SIGNAL,
 *                                K_POLL_MODE_NOTIFY_ONLY,
 *                                &log_timer_signal),
 *   };
 * ============================================================ */
void logging_task_fn(void *p1, void *p2, void *p3) {
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    system_state_t prev_state = STATE_NORMAL;

    /* TODO: Set up k_poll events and start log timer */

    while (1) {
        /* TODO: Your code here */
    }
}
```

4 Part 2: Priority Inversion Experiment

After your correct implementation works, replace `src/main.c` with the provided `src/main_priority_inversion.c`, rebuild, and run. This version uses the *same architecture* as your correct solution—lock-free alarm path, mutex-protected log snapshot, `k_poll` logging—but introduces two deliberate bugs:

- **Bug 1:** Logging task priority is set to 1 (highest!), processing task remains at 5.
- **Bug 2:** Logging task calls `k_busy_wait(150000)` (150 ms of CPU-blocking work) on every wakeup.

Why doesn't the lock-free alarm path save us? You might expect that a lock-free design cannot be blocked. That is true for **mutex contention**: the processing task's alarm path never acquires a lock, so no other task can delay it by holding a lock. But `k_busy_wait()` is not a lock—it is a **CPU scheduling** problem. While the logging task spins at priority 1, the scheduler *will not* run the processing task at priority 5, because 5 is lower priority than 1. The processing task's code is perfect; it simply never gets the CPU. Lock-free protects against data contention. Only correct priority assignment protects against scheduling inversion.

4.1 What to Observe

Run the simulation and compare the UART output to your correct implementation. **Pay special attention to the `sched=` values**—in your correct solution, they should be in the low microseconds ($\sim$10–20 μs). In the buggy version, they will spike to 150,000+ μs whenever the logging task's busy-wait overlaps with a sensor posting.

Q1: What happens when SpO2 crosses below 90%? Is the alarm activated immediately, or is there a delay? What do the `sched=` values tell you?

[Your observation]

Q2: Draw a timeline showing what each task is doing around the moment SpO2 crosses the critical threshold. Indicate where the alarm *should* fire vs. where it *actually* fires.

[Draw your timeline]

Q3: In a real hospital, what would the consequence be? A patient with SpO2 at 85% and no alarm for 150+ ms—is this acceptable? What about if the busy-wait were 500 ms?

[Your answer]

5 Reflection Questions

Q4: Compare the Week 4 architecture (bare-metal) with the Week 5 architecture (3 RTOS tasks). What can the RTOS version do that the bare-metal version cannot? What new failure modes does the RTOS introduce?

[Your answer]

Q5: The `k_msgq_put()` in the sensor task uses `K_NO_WAIT`. What happens if the queue is full? What would change if we used `K_FOREVER` instead?

[Your answer]

Q6: Why does the processing task write the alarm GPIO *before* acquiring `log_mutex`? What would happen to alarm latency if the order were reversed?

[Your answer]

Q7: Why does the sensor task use `k_timer_status_sync()` instead of `k_msleep(200)`? Draw what happens to the period in each case when I2C reads take 5 ms.

[Your answer]

6 Going Further: Toward Hard Real-Time

Our SafePulse implementation is a **firm real-time** system: a missed deadline degrades quality of service (delayed alarm), but the system continues to operate and will catch the next sample. A **hard real-time** system treats a missed deadline as a system failure—equivalent to a wrong answer—and must transition to a known-safe state. This section sketches what it would take to promote SafePulse from firm to hard real-time.

6.1 Firm vs. Hard: The Conceptual Boundary

Property	Firm Real-Time (our lab)	Hard Real-Time (production)
Missed deadline	Late result is still useful; system recovers next cycle	Late result = system failure; must enter safe state
Timing guarantee	Statistical ("usually <80 μs")	Absolute ("always < WCET bound")
Failure model	Alarm delayed; nurse may notice anyway	No second chance; must trip fail-safe
Verification	Runtime instrumentation (what we do)	Static WCET analysis + formal proof
Typical domains	Video streaming, data logging	Airbag controllers, flight control, IEC 62304 Class C

In the instrumented output from Part 1, you see lines like `[sched=12 us, proc=3 us, alarm=18 us]`. These *measurements* tell you what happened in this particular run. A hard real-time guarantee requires you to know the *worst case that could ever happen*—including under interrupt storms, cache misses, and bus contention—and to prove that the worst case still meets the deadline.

6.2 What Zephyr Provides

Zephyr offers several mechanisms toward hard real-time, though none is sufficient alone:

- **Earliest-Deadline-First (EDF) ordering:** `k_thread_deadline_set()` assigns an absolute deadline to a thread. Among threads of equal priority, the one with the nearest deadline runs first. This is useful when multiple tasks share the same criticality level but have different urgency.
- **Task watchdog API:** `k_wdt_install_timeout()` / `k_wdt_feed()` lets a task declare "I must complete within X ms." If the task fails to *feed* the watchdog in time, a callback fires. This detects deadline misses at runtime.
- **Hardware watchdog:** If the software watchdog callback itself hangs (e.g., scheduler is dead), the hardware watchdog resets the entire MCU—an ultimate fail-safe.
- **Stack overflow detection:** `CONFIG_THREAD_STACK_INFO=y` and `CONFIG_STACK_SENTINEL=y` detect when a task exceeds its allocated stack, a common source of silent corruption.

6.3 What You Would Still Need

Zephyr primitives handle detection and recovery, but hard real-time also requires **prevention**:

- **WCET analysis:** Worst-Case Execution Time must be bounded statically (using tools like aiT or OTAWA) or by measurement with sufficient margin (e.g., measured max $\times 1.5$). Our cycle-count instrumentation is a starting point but not a proof—you need to exercise every code path under worst-case conditions.
- **Scheduling analysis:** Rate Monotonic Analysis (RMA) or response-time analysis must prove that all tasks meet their deadlines simultaneously, accounting for preemption, interrupts, and blocking times.

- **Stack bounds:** Each task's stack must be provably sufficient for the deepest call chain. `-fstack-usage` can help, but recursive or indirectly-called functions require manual analysis.
- **Fail-safe policy:** The system must define what happens when a deadline is missed. For SafePulse, a reasonable policy is: trigger the alarm unconditionally (fail-safe-on) and set a sensor-fault state. A nurse will investigate—a false alarm is vastly preferable to a missed alarm.
- **No unbounded operations on the critical path:** `printk()` is particularly dangerous because UART transmission time depends on baud rate and buffer depth. On the alarm path, no output function may appear. Our lock-free alarm path already satisfies this constraint.

6.4 Code Sketch: Deadline-Aware Processing

The following sketch (not part of the lab implementation) shows how you might combine `k_thread_deadline_set()`, a task watchdog, and a fail-safe transition. This is illustrative—a production system would require formal timing analysis.

Listing 5: Sketch: absolute-release scheduling with deadline miss detection

```c
/* Hard-RT processing task (SKETCH - not for lab use) */
#define ALARM_DEADLINE_US  80   /* NFR-01 budget: 80us */
#define ALARM_DEADLINE_CYC (ALARM_DEADLINE_US *
    CONFIG_SYS_CLOCK_HW_CYCLES_PER_SEC / 1000000)

void processing_task_hard_rt(void *p1, void *p2, void *p3) {
    ARG_UNUSED(p1); ARG_UNUSED(p2); ARG_UNUSED(p3);

    /* Install task watchdog: if we don't feed within 1ms,
     * the watchdog callback fires. */
    int wdt_channel = k_wdt_install_timeout(
        wdt_dev, &(struct wdt_timeout_cfg){
            .window.max = 1,  /* 1 ms */
            .callback = processing_wdt_callback,
        });
    k_wdt_setup(wdt_dev, 0);

    while (1) {
        vitals_msg_t msg;
        k_msgq_get(&vitals_msgq, &msg, K_FOREVER);

        /* Set absolute deadline from message timestamp */
        k_thread_deadline_set(k_current_get(),
            (int)ALARM_DEADLINE_CYC);

        uint32_t proc_start = k_cycle_get_32();

        /* --- Lock-free alarm path (same as lab) --- */
        system_state_t new_state = evaluate_thresholds(&msg);

        if (new_state == STATE_CRITICAL) {
            alarm_on();
        }

```

```
        /* Check: did we exceed the deadline? */
        uint32_t elapsed_us = k_cyc_to_us_floor32(
            k_cycle_get_32() - msg.sensor_cycles);

        if (elapsed_us > ALARM_DEADLINE_US) {
            /* DEADLINE MISS --- transition to fail-safe.
             * Alarm is already on (fail-safe-on policy).
             * Do NOT call printk here --- it would make
             * the overrun worse and is itself unbounded. */
            current_state = STATE_SENSOR_FAULT;
            /* Log the miss via a flag; logging task
             * will report it on its own schedule. */
            deadline_miss_count++;
        } else {
            current_state = new_state;
        }

        /* Feed the watchdog --- we completed in time */
        k_wdt_feed(wdt_dev, wdt_channel);

        /* Log path (same as lab) */
        update_log_snapshot(&msg, new_state, elapsed_us);
    }
}
```

Why `printk` cannot appear in the fault path: `printk()` writes to UART, whose transmission time depends on baud rate (at 115200 baud, a 60-character line takes ∼5.2 ms). If the task is *already* past its deadline and then calls `printk()`, the overrun grows by milliseconds. In a hard-RT fault handler, you may only: set a GPIO (the alarm), update a volatile flag, and feed or skip the watchdog. Diagnostics are the logging task's job—it runs at low priority, on its own time.

6.5 The Gap Between This Lab and Production

Aspect	This lab	Production hard-RT
Timing verification	Runtime measurement	Static WCET + RMA proof
Deadline detection	Cycle-count logging	Hardware watchdog + EDF
Fail-safe policy	None (alarm is "best effort")	Defined, tested, certified
UART on critical path	Excluded (lock-free design)	Excluded + formally verified
Stack analysis	Fixed 1024 B guess	`-fstack-usage` + margin
Certification	N/A	IEC 62304 Class C / DO-178C Level A

The lab gives you the *architectural foundation*—lock-free alarm paths, priority separation, instrumented timing—that production hard-RT systems build on. The remaining gap is primarily in **verification**: proving that the architecture works under all conditions, not just the ones you tested.

7 Looking Ahead — Week 6

You now have a working multi-task RTOS system with correct priority assignment, lock-free alarm paths, and cycle-accurate instrumentation. Next week, you will write **automated tests** using Robot Framework that validate your implementation against the requirements from Week 2. You will inject specific sensor values, observe system state transitions, and verify timing—closing the loop from specification to implementation to validation.

> **Full circle:** In Week 2, you wrote requirements. In Weeks 3–5, you implemented them. In Week 6, you will **prove** they are met. This is the V-model of embedded systems development.